

Programming with OpenMP

Danni Noren dannin@kth.se

Erik Sevelin esevelin@kth.se

February 2025

Introduction

In this assignment, we investigate the parallel execution of matrix operations as well as quicksort using openMP. We analyze the performance over different processors and input sizes.

Matrix Summation

Implementation

Here, we used OpenMP to parallelize matrix summation along with finding maximum and minimum values and their positions in the matrix. Multiple processors work on different part of the matrix, each processor computing its local minimum, maximum and partial sum. The global variables are updated safely with the values from each processor. The directive `#pragma omp for nowait` allows the loop iterations to continue without forcing the processors to wait. Additionally `#pragma omp critical` ensures that the global values are updated safely preventing race condition. Finally the program prints execution time along with the computed minimum and maximum values, their positions and the total sum.

Results

Matrix Size	Threads	Median Execution Time (s)	Speedup
500x500	1 (Sequential)	0.0008	1.0
	2	0.0014	0.5714
	3	0.0018	0.4444
	4	0.0018	0.4444
	5	0.0022	0.3636
1000x1000	1 (Sequential)	0.0016	1.0
	2	0.0018	0.8888
	3	0.0022	0.7272
	4	0.0022	0.7272
	5	0.0032	0.5
5000x5000	1 (Sequential)	0.0366	1.0
	2	0.0242	1.5
	3	0.0206	1.8
	4	0.0162	2.3
	5	0.0228	1.6

Table 1: Performance with Different Matrix Sizes and Threads

Conclusion

As seen in the tables above, for smaller matrix such as 500x500, there was no significant speedup presumably due to overhead. However, for larger matrices, such as 5000x5000 in the table above we observe a notable improvement in the execution time, specially when using 4 threads. The performance declines when we use t threads instead, this can be due to overhead and memory contention. In conclusion, OpenMP works best for larger inputs where the benefits of parallelization outweighs the overhead.

Quicksort

Implementation

Using OpenMP on a previously implemented Java parallel quicksort program which was reprogrammed in C, there is no need to explicitly manage threads and keep track of how many threads are active at any time. OpenMP divides each partition of the quicksort() algorithm into tasks. These tasks are stored to be executed whenever a thread is available, making task distribution highly efficient.

Results

Array Size	Threads	Median Execution Time (ms)	Speedup
1000	1 (Sequential)	0.0819	1.0
	2	0.3324	4.0586
	3	0.4214	5.1453
	4	0.4736	5.7827
	5	0.6645	8.1136
10 000	1 (Sequential)	1.0848	1.0
	2	0.7441	0.6859
	3	0.7631	0.7034
	4	0.7309	0.6738
	5	0.8657	0.7980
100 000	1 (Sequential)	10.5995	1.0
	2	6.2410	0.5888
	3	4.9644	0.4684
	4	4.8923	0.4616
	5	4.2671	0.4026

Table 2: Performance of Parallel Quicksort

Conclusion

Programming with OpenMP was very efficient and reduced the amount of code that was required to be written for the program to work.

For the array size of 1000 elements, the time is increased as threads are increased. This is most likely caused by threads being rather demanding, and pay off when larger amounts of data are processed in parallel. For the array size of 10 000 the results show less distance between each other and in general are more efficient than executing the program using a single thread. Once we get to an array size of 100 000 the threads get to shine and it's clear that the more threads we add, the faster the program finishes.